HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO PROJECT 01

DIABETES AGENT

HỆ ĐA TÁC TỬ HỖ TRỢ SÀNG LỌC VÀ TRA CỨU KIẾN THỨC TIỂU ĐƯỜNG HỌC SÂU, TRI THỨC VÀ HỌC TĂNG CƯỜNG

Họ và tên:	Bùi Anh Đức
Lớp:	D23CTPM01
Mã sinh viên:	B23DCCN165
Học kỳ:	I năm học 2026 – 2027
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế

Hà Nội, tháng 11/2026

MỤC LỤC

1 Tổng quan đề tài **1**

1.1 Bài toán 1

1.2 Các chức năng chính 1

1.3 Phạm vi triển khai 1

2 Cơ sở tác tử và kiến trúc hệ thống **2**

2.1 Tác tử và hệ đa tác tử 2

2.2 Kiến trúc tổng thể 2

2.3 Vai trò và hợp đồng đầu ra 3

2.4 Thông điệp và điều kiện dừng 4

2.5 Luật ưu tiên và quyền kiểm tra 4

3 Dữ liệu và mô hình học sâu **5**

3.1 Nguồn dữ liệu và ý nghĩa nhãn 5

3.2 Đặc trưng và mã hóa 5

3.3 Chia tập và chống rò rỉ 6

3.4 MLP PyTorch và hàm mất mát 6

3.5 Các mô hình đối chứng 7

3.6 Ngưỡng và chỉ số đánh giá 7

4 Tri thức và học tăng cường **8**

4.1 Quy trình xây dựng kho tri thức 8

4.2 Đồ thị khái niệm local 8

4.3 Truy xuất và kết xuất câu trả lời 9

4.4 Thiết kế đánh giá truy xuất 10

4.5 MDP nhắc ghi nhật ký 10

4.6 Huấn luyện Q-learning 10

5 Cài đặt và giải thích mã nguồn **11**

5.1 Tổ chức mã nguồn 11

5.2 Đọc dữ liệu và kiểm tra chia tập 11

5.3 Mạng MLP và chuẩn hóa đầu vào 12

5.4 Vòng huấn luyện và lưu checkpoint 13

5.5 Chọn ngưỡng và mô hình mặc định 13

5.6 Suy luận từ mô hình đã lưu 14

5.7 Truy xuất tri thức 15

5.8 Cập nhật bảng Q 16

5.9 Giao tiếp và kiểm tra kết quả tác tử 16
5.10 Điều phối một lượt xử lý 18
5.11 Liên kết mã nguồn với giao diện 19
5.12 Notebook và các tệp kết quả 20

6 Thực nghiệm và phân tích kết quả 21
6.1 Điều kiện thực nghiệm 21
6.2 Kết quả validation và lựa chọn mặc định 21
6.3 Kết quả trên test đã khóa 22
6.4 Quá trình học của MLP 23
6.5 Truy xuất tri thức và vai trò đồ thị 23
6.6 Kết quả Q-learning và đối chứng 24
6.7 Kiểm tra tích hợp và lỗi được inject 25
6.8 Kiểm thử tự động và notebook 25

7 Giao diện và kết quả chạy demo 26
7.1 Dashboard tổng quan 26
7.2 Kết quả phân loại và phối hợp tác tử 27
7.3 Tra cứu tri thức và bản đồ khái niệm 28
7.4 Biểu đồ thực nghiệm trên giao diện 29

8 Môi trường và cách chạy hệ thống 30
8.1 Cài đặt và khởi động local 30
8.2 Tái lập thực nghiệm và notebook 30
8.3 Dữ liệu và lưu kết quả 31
8.4 Các lỗi vận hành thường gặp 31
8.5 Biên dịch báo cáo 31

9 Kết luận và hướng phát triển 32
9.1 Kết quả đạt được 32
9.2 Giới hạn và hướng phát triển 32

Tài liệu tham khảo 33

DANH MỤC HÌNH

2.1 Các nguồn thông tin và cơ chế phối hợp của Diabetes Agent. 3

4.1 Đồ thị 12 khái niệm và các quan hệ được biên soạn trong project. Các ghi chú D01–D12 được gắn cạnh bằng chứng trong JSON. 9

6.1 ROC và Precision–Recall trên test. Mọi quyết định về model/ngưỡng được thực hiện bằng validation trước đó. 22

6.2 Loss train và validation, đường dọc thể hiện epoch được chọn để lưu checkpoint. 23

6.3 Đường học năm seed và reward đánh giá trong môi trường nhắc việc giả lập. . 24

7.1 Dashboard tổng quan và các nút chạy nhanh của Diabetes Agent. 26

7.2 Kết quả hội chẩn với score, ngưỡng phân loại và trạng thái tác tử. 27

7.3 Kết quả truy xuất theo câu hỏi và đồ thị khái niệm trên giao diện. 28

7.4 Biểu đồ tương tác so sánh mô hình phân loại và chính sách nhắc việc. 29

DANH MỤC BẢNG

2.1 Liên hệ loại tác tử và hiện thực trong Project01. 2
2.2 Bảy vai trò tính cả điều phối. 3
2.3 Ba performative được hiện thực thực tế. 4

3.1 Nhóm đặc trưng và những điểm dễ nhập sai. 5
3.2 Phân bố tập nghiên cứu được sử dụng thực tế. 6
3.3 Cấu hình huấn luyện MLP được khóa trong source. 7

4.1 Danh mục 12 ghi chú có liên kết tới bảy trang nguồn. 8

5.1 Nội dung và đầu ra của bốn notebook. 20

6.1 Chỉ số trên validation; ngưỡng được chọn bằng F1 validation. 21
6.2 Chỉ số trên test; ngưỡng được chọn bằng F1 validation. 22
6.3 Ngưỡng đã khóa và số lượng dự đoán trên test. 22
6.4 Đánh giá truy xuất trên 24 câu hỏi của bộ thực nghiệm. 23
6.5 Reward mỗi ngày trong simulator; std giữa năm giá trị trung bình theo seed train. 24
6.6 Các kịch bản chức năng được chạy ở phiên bản bàn giao. 25

8.1 Biểu hiện lỗi và thao tác khắc phục. 31

DANH MỤC MÃ NGUỒN

5.1 Đọc CSV và kiểm tra tính toàn vẹn dữ liệu. 11
5.2 Kiến trúc MLP sử dụng trong huấn luyện và suy luận. 12
5.3 Chuẩn hóa dữ liệu và khởi tạo bộ tối ưu. 12
5.4 Huấn luyện minibatch và chọn checkpoint theo validation. 13
5.5 Chọn ngưỡng phân loại bằng F1 validation. 13
5.6 Tổng hợp validation và lưu quyết định chọn mô hình. 14
5.7 Hàm dự đoán của lớp RiskModel. 14
5.8 TF-IDF kết hợp mở rộng khái niệm một bước. 15
5.9 Huấn luyện Q-learning cho tác tử nhắc nhật ký. 16
5.10 Gửi yêu cầu và ghi thông điệp phản hồi qua MessageBus. 16
5.11 Các điều kiện kiểm tra của Council. 17
5.12 Nhánh tiếp nhận, gọi chuyên gia và tổng hợp của Coordinator. 18
5.13 Callback của ba kịch bản chạy nhanh. 19
5.14 Dựng trạng thái hiển thị của bảy vai trò tác tử. 19
8.1 Cài thư viện và khởi động giao diện local. 30
8.2 Tái lập artifact, notebook và kiểm thử. 30

Phần 1

Tổng quan đề tài

1.1 Bài toán

Diabetes Agent tiếp nhận hồ sơ sức khỏe và câu hỏi về tiểu đường, sau đó trả kết quả phân loại, nội dung tra cứu và hành động nhắc ghi nhật ký. Ba nhóm công việc sử dụng ba nguồn thông tin: dữ liệu khảo sát CDC, kho ghi chú NIDDK/CDC và môi trường học tăng cường. Các tác tử chuyên trách xử lý từng nhánh trước khi kết quả được kiểm tra và tổng hợp.

Đề tài triển khai luồng CSDL–học sâu, KB–truy xuất và RL–luật theo cấu trúc tham khảo trong báo cáo Project01 của Trần Văn Khánh [13]. Số liệu trong báo cáo này lấy từ các lần chạy của Project01, lưu tại `artifacts/`. Tập nghiên cứu được kế thừa từ `Diabetes_MultiAgent`; các mô hình phân loại của Project01 được huấn luyện và lưu riêng.

1.2 Các chức năng chính

Chức năng phân loại nhận 21 chỉ báo sức khỏe, kiểm tra mã đầu vào rồi chạy mô hình đã lưu. Chức năng tra cứu tìm các ghi chú liên quan đến câu hỏi và trả nội dung kèm nguồn. Chức năng nhắc nhật ký chọn hành động từ bảng Q theo số ngày chưa ghi và trạng thái mệt vì thông báo. Tác vụ tổng hợp kết hợp cả ba chức năng trong một lượt xử lý.

Coordinator phân công công việc qua MessageBus. Intake kiểm tra yêu cầu trước khi gọi chuyên gia; Council kiểm tra các kết quả trả về; Advisor ghép nội dung cuối cùng. Giao diện Streamlit hiển thị kết quả và trạng thái của bảy vai trò, đồng thời xuất vết xử lý dưới dạng JSON.

1.3 Phạm vi triển khai

Bài toán phân loại sử dụng nhãn nhị phân của CSV UCI: lớp 1 gộp tiền tiểu đường/tiểu đường, lớp 0 không có nhãn này. Kho tri thức gồm các ghi chú tiếng Việt biên soạn từ bảy trang NIDDK/CDC [6, 7]. Nội dung trả lời được trích xuất từ ghi chú tìm được.

Q-learning được áp dụng cho lịch nhắc nhật ký trong một môi trường mô phỏng có tám trạng thái và ba hành động. Các tác tử chạy đồng bộ trong một tiến trình Python. Giao diện sử dụng dữ liệu, mô hình và kho tri thức đã đóng gói tại máy local.

Phần 2

Cơ sở tác tử và kiến trúc hệ thống

2.1 Tác tử và hệ đa tác tử

Trong cách tiếp cận tác tử, một thành phần quan sát môi trường, lựa chọn hành động theo mục tiêu và cung cấp phản hồi để các thành phần khác tiếp tục xử lý. Các tính chất tự chủ, phản ứng, chủ động và tương tác thường được dùng để phân tích agent [1]. Mức tự chủ của Project01 có giới hạn: tác tử có thể yêu cầu dữ liệu, từ chối khi thiếu bằng chứng hoặc trả FAILURE, nhưng không tự chọn công cụ ngoài tập được lập trình.

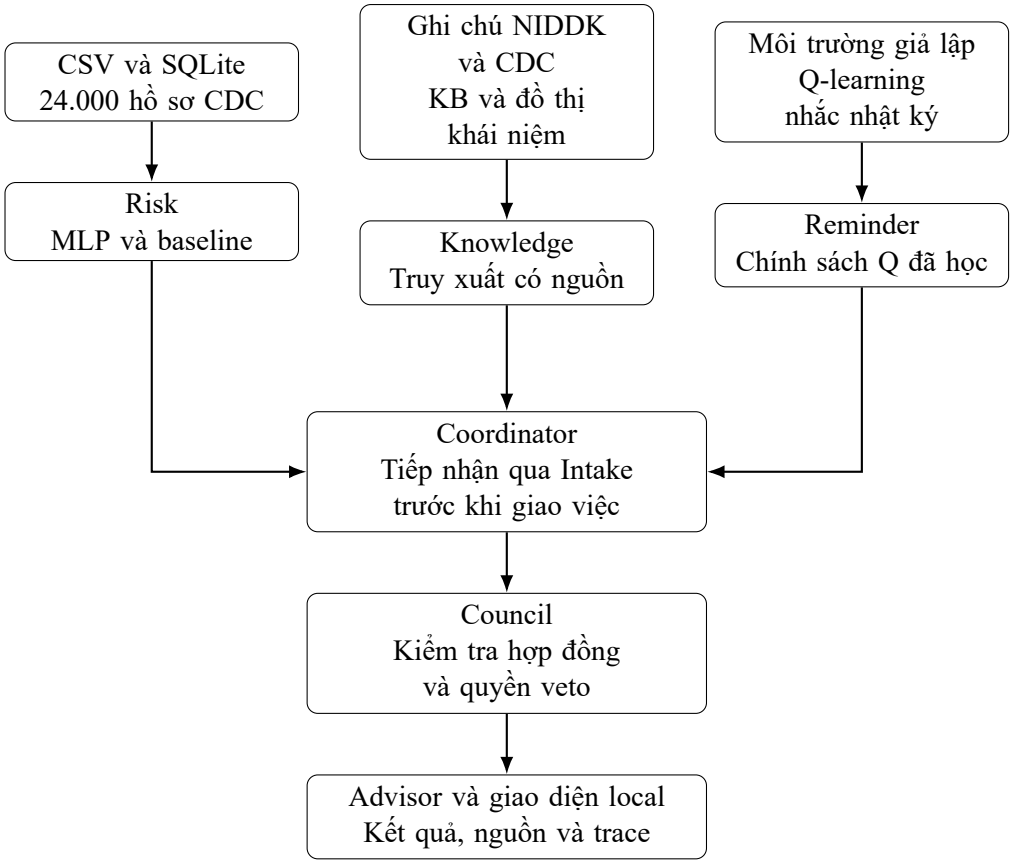
Hệ đa tác tử phân chia bài toán theo trách nhiệm và phối hợp qua một giao thức. Risk sử dụng mô hình đã học; Knowledge sử dụng kho tri thức; Reminder sử dụng Q-table; Intake và Council áp dụng luật. Các vai trò có cách ra quyết định khác nhau và được kết nối bởi Coordinator.

Bảng 2.1: Liên hệ loại tác tử và hiện thực trong Project01.

Loại cơ chế	Hiện thực và giới hạn
Phản ứng theo luật	Intake ưu tiên dấu hiệu khẩn cấp và phạm vi; chỉ bao phủ điều kiện đã mã hóa.
Dựa trên mô hình học	Risk nạp MLP hoặc baseline, suy luận trên vector khảo sát hợp lệ.
Dựa trên tri thức	Knowledge truy xuất ghi chú và liên kết khái niệm; không suy diễn lâm sàng mở.
Học từ phần thưởng	Reminder chọn argmax Q theo trạng thái mô phỏng; không tự huấn luyện trong request.
Kiểm tra và tổng hợp	Council kiểm tra hợp đồng; Advisor ghép kết quả có nguồn theo mẫu.

2.2 Kiến trúc tổng thể

Hình 2.1 thể hiện ba nhánh dữ liệu cùng hội tụ ở điều phối. Dữ liệu nghiên cứu và tri thức được chuẩn bị trước, còn mỗi request chỉ thực hiện kiểm tra, suy luận, truy xuất và tổng hợp. Không có thao tác fit trong luồng request, nên việc dự đoán của demo tương ứng với artifact đã đánh giá.



Hình 2.1: Các nguồn thông tin và cơ chế phối hợp của Diabetes Agent.

2.3 Vai trò và hợp đồng đầu ra

Bảng 2.2: Bảy vai trò tính cả điều phối.

Vai trò	Đầu vào chính	Đầu ra và trách nhiệm
Coordinator	Tác vụ, câu hỏi, hồ sơ	Giao việc theo tác vụ; dừng sớm nếu Intake không cho phép.
Intake	Câu hỏi và context	completed, urgent, needs_input hoặc out_of_scope.
Risk	21 input và model ID	Score, threshold, nhãn; liệt kê thiếu/sai mã khi chưa suy luận được.
Knowledge	Câu hỏi	Tối đa ba ghi chú, ID/URL và câu trả lời; có thể thiếu bằng chứng.
Reminder	Gap và fatigue	Trạng thái, ba Q-value và hành động nhắc việc.
Council	Các proposal	approved/rejected cùng lý do; không sửa điểm model.
Advisor	Proposal được duyệt	Ghép kết quả và các trích đoạn có nguồn.

Các tác vụ được chọn trên giao diện gồm consult, knowledge, risk và reminder. consult gọi cả ba nhánh chuyên môn, trong khi các tác vụ riêng chỉ gọi thành phần cần thiết.

2.4 Thông điệp và điều kiện dừng

MessageBus nhận sender, receiver, task và payload; ghi REQUEST trước khi gọi handler, sau đó ghi INFORM hoặc FAILURE. Mỗi thông điệp phản hồi có trạng thái và thời gian xử lý. Khi handler phát sinh exception, bus chuyển lỗi thành kết quả failure để Council có thể từ chối tổng hợp. Thông tin lỗi này dùng cho demo kỹ thuật local.

Một lượt consult bình thường có sáu cặp request/response, tương ứng 12 bản ghi: Intake, Risk, Knowledge, Reminder, Council và Advisor. Nếu Intake phát hiện trạng thái urgent, hệ thống kết thúc sau hai thông điệp và không gọi Risk. Nếu Council bác proposal, Advisor không được gọi.

Bảng 2.3: Ba performative được hiện thực thực tế.

Loại	Ý nghĩa
REQUEST	Coordinator yêu cầu một handler thực hiện nhiệm vụ với payload cụ thể.
INFORM	Handler trả trạng thái và dữ liệu. approved/rejected cũng là nội dung INFORM từ Council.
FAILURE	Handler không thực hiện được do exception; chưa thể coi là kết quả thành công.

Giao thức REQUEST/INFORM/FAILURE được cài đặt trong một tiến trình và gọi đồng bộ. Trace ghi thứ tự thực thi, kết quả và thời gian của từng handler.

2.5 Luật ưu tiên và quyền kiểm tra

Intake được thực hiện trước chuyên gia. Những cụm dấu hiệu như bất tỉnh, co giật hoặc khó thở nặng làm hệ thống chuyển nhánh cần hỗ trợ y tế. Số đo glucose bổ sung phải hữu hạn, đúng khoảng kiểm tra và được nhập theo mg/dL; dưới 70 mg/dL kích hoạt cảnh báo có nguồn CDC [10]. Đây là luật cảnh báo riêng, không phải nhãn do mô hình khảo sát suy ra.

Council kiểm tra ID bằng chứng, sự hiện diện của nguồn khi Knowledge trả completed, khoảng score và các lỗi failure. Proposal loại medication bị bác. Các điều kiện này xác định hợp đồng đầu ra của hệ thống.

Phần 3

Dữ liệu và mô hình học sâu

3.1 Nguồn dữ liệu và ý nghĩa nhãn

Nguồn công khai là CDC Diabetes Health Indicators trên UCI [3]. CSV nguồn có 253.680 dòng với 21 đặc trưng và cột nhãn nhị phân. Project sử dụng mẫu nghiên cứu cố định gồm 24.000 dòng. Metadata lưu cả SHA256 nguồn và SHA256 mẫu, URL và quy trình chia tập; hàm đọc dừng nếu checksum mẫu không còn khớp.

Nhãn được đổi tên từ `Diabetes_binary` thành `target`. Trong diễn giải của project, lớp 1 là nhãn gộp tiền tiểu đường/tiểu đường theo mã UCI, còn lớp 0 là không có nhãn đó trong dữ liệu. Nhãn mô tả trạng thái khảo sát gộp, không phân biệt riêng type 2 hoặc thời điểm phát bệnh. Trang UCI có phần giới thiệu chưa thống nhất với bảng số đặc trưng; project bám schema của CSV thực tế và ghi nhận khác biệt trong provenance.

3.2 Đặc trưng và mã hóa

Các biến bao gồm huyết áp/cholesterol cao, BMI, hút thuốc, tiền sử đột quỵ hoặc bệnh tim, vận động, rau/trái cây, bảo hiểm và rào cản chi phí, tự đánh giá sức khỏe, số ngày sức khỏe kém, khó đi lại, giới tính và các nhóm tuổi/học vấn/thu nhập.

Bảng 3.1: Nhóm đặc trưng và những điểm dễ nhập sai.

Nhóm	Mã và ý nghĩa trong project
Biến nhị phân	0/1; không tự biến câu trả lời mơ hồ thành một giá trị mặc định.
BMI	Số liên tục trong khoảng 12–98, đơn vị kg/m^2.
Age	Mã 1–13, không phải tuổi theo năm; 1 ứng với 18–24, 13 từ 80 tuổi.
GenHlth	Mã 1–5, từ sức khỏe tự đánh giá tốt tới kém.
MentHlth và PhysHlth	Số ngày 0–30 trong giai đoạn khảo sát.
Education và Income	Các nhóm 1–6 và 1–8 theo schema Hoa Kỳ; không tự quy đổi thành thước đo Việt Nam.
Cột quản trị	target, row_id, group và split không được đưa vào input.

Các mã có thứ tự được đưa vào dưới dạng số như một lựa chọn đơn giản. Điều đó áp đặt quan hệ khoảng cách cho Logistic/MLP và là một giới hạn; project chưa so sánh one-hot encoding hoặc mô hình chuyên biệt cho biến phân loại. Tất cả model dùng cùng tập đặc trưng để giữ điều kiện đối chiếu.

3.3 Chia tập và chống rò rỉ

Bảng 3.2: Phân bố tập nghiên cứu được sử dụng thực tế.

Tập	Số dòng	Nhãn 1	Tỷ lệ nhãn 1
train	16787	2375	14.15%
test	3612	496	13.73%
validation	3601	473	13.14%

Split được kế thừa nguyên từ tập nghiên cứu, với seed 42/43 ở bước chia theo nhóm. Mỗi nhóm được định nghĩa bởi fingerprint của cùng vector 21 đặc trưng. Các fingerprint giống nhau không xuất hiện ở hai tập khác nhau. Hàm `load_data` còn tính lại fingerprint để không chỉ tin cột group được lưu sẵn.

Dữ liệu thiếu định danh bệnh nhân, nên phép chia chỉ kiểm soát các vector trùng nhau giữa các split. Phân bố nhãn giữa các split có chênh lệch nhỏ do chia theo nhóm, không ép cân bằng thành 50/50.

Scaler chỉ fit trên train. Validation được dùng để chọn epoch, model mặc định và ngưỡng. Test không được dùng để chọn lại các quyết định đó. Việc chạy lại notebook sử dụng cùng source, seed và split; các lượt lặp để xác nhận tái lập không được coi là các lần thử độc lập.

3.4 MLP PyTorch và hàm mất mát

Mạng sử dụng hai lớp ẩn kích thước 64 và 32, ReLU, dropout 0,15 sau lớp đầu và một logit đầu ra. Với vector đầu vào đã chuẩn hóa x, mô hình tính:

$$h_1 = \mathrm{ReLU}(W_1 x + b_1), \tag{3.1}$$

$$h_2 = \mathrm{ReLU}(W_2\,\mathrm{Dropout}(h_1) + b_2), \tag{3.2}$$

$$z = W_3 h_2 + b_3, \qquad p = \sigma(z). \tag{3.3}$$

Số tham số học là $21 \cdot 64 + 64 + 64 \cdot 32 + 32 + 32 + 1 = 3.521$. Đầu ra logit được đưa trực tiếp vào `BCEWithLogitsLoss`; sigmoid chỉ áp dụng khi suy luận. Việc kết hợp logit và BCE trong cùng toán tử giúp ổn định tính toán theo tài liệu PyTorch [4].

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \sigma(z_i) + (1 - y_i)\log(1 - \sigma(z_i))\right]. \tag{3.4}$$

Huấn luyện giữ phân bố nhãn gốc, không dùng trọng số lớp hoặc tái lấy mẫu. Brier score đo sai số của score trên test nội bộ; calibration trên quần thể mục tiêu chưa được đánh giá.

Bảng 3.3: Cấu hình huấn luyện MLP được khóa trong source.

Thông số	Giá trị
Kiến trúc	21–64–32–1
Optimizer	Adam; learning rate 0,001; weight decay 10^{-4}
Batch và epoch tối đa	256; 80
Early stopping	BCE validation; patience 10; mức cải thiện 10^{-5}
Khởi tạo	Seed 42; CPU; 2 thread PyTorch
Checkpoint	Weight có BCE validation thấp nhất, không phải epoch cuối

3.5 Các mô hình đối chứng

Logistic Regression là pipeline StandardScaler và bộ phân loại tuyến tính với tối đa 1.000 vòng lặp. Random Forest dùng 160 cây, ít nhất 8 mẫu ở lá, max_features bằng căn bậc hai và seed 42. DummyClassifier với strategy prior luôn trả phân bố nhãn train. Đây là các cấu hình cố định của project, chưa thực hiện tìm kiếm siêu tham số toàn diện.

Các mô hình dùng cùng split và đặc trưng để đối chiếu kết quả trên dữ liệu bảng. Model mặc định là mô hình có Average Precision validation cao nhất trong nhóm MLP, Logistic Regression và Random Forest.

3.6 Ngưỡng và chỉ số đánh giá

Mỗi model chọn ngưỡng từ 151 ứng viên đều nhau trong khoảng 0,05–0,80, tối đa hóa F1 validation; nếu bằng điểm, chọn ứng viên xuất hiện đầu tiên. Sau đó ngưỡng được giữ nguyên khi đánh giá test. Cách này phục vụ benchmark kỹ thuật; không tối ưu chi phí lâm sàng của âm tính giả/dương tính giả.

ROC AUC mô tả khả năng xếp hạng giữa hai lớp. Average Precision tổng hợp precision theo các mức recall và phù hợp để quan sát bài toán lớp dương ít [5]. Precision, recall, specificity, F1 và ma trận nhầm lẫn được báo tại ngưỡng đã chọn. Brier là trung bình $(p_i - y_i)^2$, thấp hơn là tốt hơn trong cùng điều kiện dữ liệu.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{3.5}$$

$$\text{F1} = \frac{2PR}{P + R}, \qquad \text{Specificity} = \frac{TN}{TN + FP}. \tag{3.6}$$

Accuracy không được dùng làm kết luận chính vì phần lớn mẫu thuộc lớp 0. Đường chuẩn Average Precision của bộ dự đoán không có khả năng phân biệt bằng tỷ lệ lớp 1 trong tập đánh giá.

Phần 4

Tri thức và học tăng cường

4.1 Quy trình xây dựng kho tri thức

Kho tri thức gồm 12 ghi chú tiếng Việt biên soạn ngắn từ bảy trang chính thức NIDDK và CDC. Mỗi ghi chú lưu ID, tiêu đề, từ khóa, khái niệm, nội dung, tên nguồn, URL và ngày kiểm tra. Nội dung được biên soạn theo từng chủ đề và lưu local.

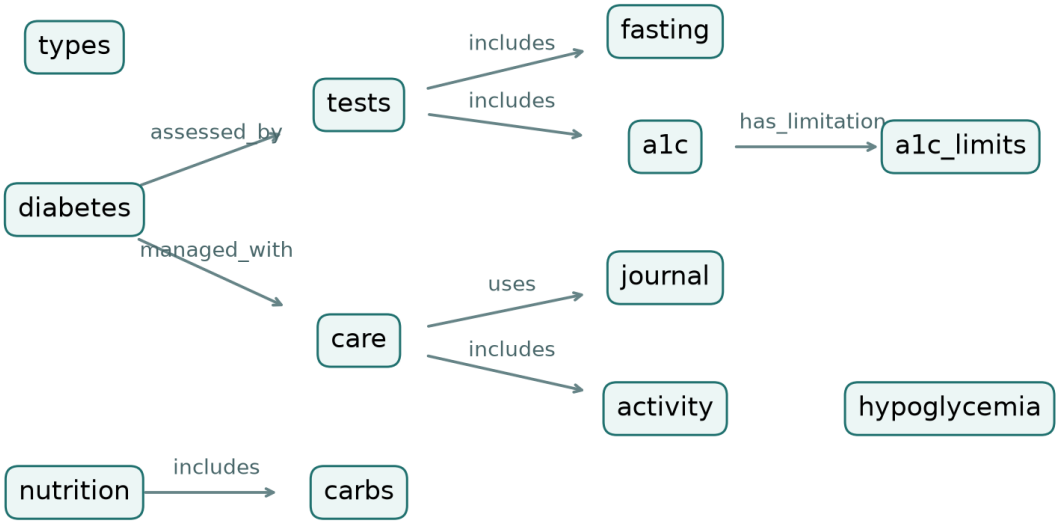
Quy trình gồm chọn chủ đề có nguồn, đọc nội dung chính thức, viết đoạn tóm lược theo phạm vi project, gán concept và liên kết nguồn, sau đó đánh giá khả năng tìm lại đoạn tương ứng. Các nguồn về xét nghiệm, HbA1c, quản lý bệnh, dinh dưỡng và vận động được liệt kê trong tài liệu tham khảo [7, 8, 9, 11, 12].

Bảng 4.1: Danh mục 12 ghi chú có liên kết tới bảy trang nguồn.

ID	Chủ đề	Nguồn
D01	Tiểu đường và insulin	NIDDK
D02	Các loại tiểu đường	NIDDK
D03	Xét nghiệm và xác nhận	NIDDK
D04	Glucose lúc đói	NIDDK
D05	Đường huyết thấp	CDC
D06	HbA1c phản ánh thời gian dài	NIDDK
D07	Giới hạn của HbA1c	NIDDK
D08	Nhật ký theo dõi	NIDDK
D09	Mục tiêu và thuốc	NIDDK
D10	Ăn uống và khẩu phần	CDC
D11	Carbohydrate	CDC
D12	Vận động	CDC

4.2 Đồ thị khái niệm local

Đồ thị có 12 khái niệm, 8 quan hệ có hướng giữa khái niệm và 12 cạnh nối ghi chú tới khái niệm được giải thích. Ví dụ, tests nối tới fasting và a1c; a1c nối tới a1c_limits. Cấu trúc này làm tường minh một số quan hệ giữa các phần tri thức và cung cấp một bước mở rộng khi truy xuất.



Hình 4.1: Đồ thị 12 khái niệm và các quan hệ được biên soạn trong project. Các ghi chú D01–D12 được gắn cạnh bằng chứng trong JSON.

Đồ thị được lưu bằng JSON và nạp khi ứng dụng khởi động. Tệp `concept_graph.cypher` chứa các nút và cạnh khái niệm ở dạng lệnh Cypher; luồng local sử dụng trực tiếp JSON.

4.3 Truy xuất và kết xuất câu trả lời

Văn bản được chuẩn hóa chữ thường và bỏ dấu, sau đó vector hóa TF-IDF với unigram/bigram. Corpus đầu vào cho từng ghi chú gồm tiêu đề, từ khóa và nội dung. Điểm lexical là cosine similarity giữa query và mỗi ghi chú. Alias khớp theo ranh giới từ giúp xác định các concept được nhắc trực tiếp.

Khi bật đồ thị, hệ thống mở rộng một bước theo cạnh có hướng. Ghi chú thuộc các concept trực tiếp hoặc mở rộng được cộng 0,12 vào điểm lexical. Để tránh suy ra bằng chứng chỉ vì ở gần trong graph, ứng viên phải có cosine ít nhất 0,10 nếu không khớp concept trực tiếp. Ngoài ra, ứng viên cần khớp alias hoặc có ít nhất hai token query ở tiêu đề/từ khóa sau khi bỏ một số từ chức năng.

$$\text{score}(d, q) = \cos(v_d, v_q) + 0,12 \mathbb{1}\{C_d \cap C_q^+ \neq \varnothing\}. \tag{4.1}$$

Các điều kiện trên là heuristic được định nghĩa trong source; chưa có tập phát triển độc lập để chứng minh ngưỡng tối ưu. Chúng ngăn một số kết quả do trùng từ thông thường nhưng cũng có thể làm bỏ sót cách diễn đạt mới.

Tối đa ba ghi chú được trả về theo điểm, nếu bằng điểm thì theo ID. Không có ứng viên đạt thì trả insufficient_evidence. Câu trả lời nối trực tiếp nội dung ghi chú cùng ID, còn giao diện hiển thị URL tương ứng. Đây là RAG theo kiểu truy xuất và kết xuất trích đoạn theo mẫu, không dùng LLM hoặc embedding model. Council kiểm tra tính hợp lệ của ID nhưng không chứng minh độ liên quan ngữ nghĩa của từng đoạn.

4.4 Thiết kế đánh giá truy xuất

Bộ đánh giá có 24 câu do tác giả soạn, mỗi ghi chú hai câu và một ID đích. Hit@1 kiểm tra kết quả đầu; Hit@3 kiểm tra ID xuất hiện trong ba kết quả; MRR lấy trung bình nghịch đảo thứ hạng, bằng 0 nếu không tìm thấy. Cùng bộ câu được chạy có và không có bonus đồ thị để xem đóng góp của thành phần này.

Bộ câu sử dụng từ vựng gần KB và đã được dùng trong quá trình phát triển. Kết quả đo khả năng tìm lại ID trong các trường hợp này; chưa có bộ câu hỏi độc lập.

4.5 MDP nhắc ghi nhật ký

Nhánh RL học chính sách cân bằng việc có bản ghi với chi phí và trạng thái mệt do thông báo. Các hệ số phản ứng và phần thưởng được đặt trong simulator.

Trạng thái có dạng $s = 2g + f$, trong đó $g \in \{0, 1, 2, 3\}$ là số ngày chưa ghi, được chặn tại 3, và $f \in \{0, 1\}$ là trạng thái mệt vì thông báo. Có 8 trạng thái và 3 hành động: $a = 0$ không nhắc, $a = 1$ nhắc ghi nhật ký, $a = 2$ gửi gợi ý ghi chép ngắn. Reminder chỉ quyết định hành động trong tập này.

Xác suất ghi nhận ở bước tiếp theo được mô phỏng:

$$P(L = 1 \mid g, f, a) = \text{clip}(0,40 - 0,07g - 0,08f + b_a;\ 0,05;\ 0,95), \tag{4.2}$$

với $b_a \in \{0; 0,35; 0,22\}$. Nếu có bản ghi, $g' = 0$; nếu không, $g' = \min(3, g + 1)$. Xác suất mệt ở ngày tiếp theo là 0,15; 0,70 hoặc 0,40 theo hành động tương ứng. Reward:

$$r = L - c_a - 0,10g' - 0,12f \mathbb{1}\{a > 0\}, \qquad c_a \in \{0; 0,22; 0,10\}. \tag{4.3}$$

Reward đo utility của chính sách theo các hệ số trên.

4.6 Huấn luyện Q-learning

Theo quy tắc Q-learning [2], tác tử cập nhật:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \tag{4.4}$$

Project dùng $\alpha = 0,08$, $\gamma = 0,92$, ϵ-greedy với $\epsilon_e = \max(0,03; 0,8 \cdot 0,997^e)$. Mỗi seed train có 1.800 episode, mỗi rollout 28 ngày. Đây là bài toán continuing: mốc 28 ngày chỉ là cửa sổ lấy mẫu, không phải trạng thái bệnh kết thúc, nên bước cuối vẫn bootstrap từ Q của trạng thái kế.

Năm seed train là 42–46. Mỗi policy được đánh giá greedy bằng 200 seed 10000–10199, tách khỏi seed train. Các policy đối chứng gồm không bao giờ nhắc, luôn nhắc và luật chỉ nhắc khi gap từ 2 và chưa mệt. Các lượt eval dùng cùng seed giữa policy để giảm nhiễu so sánh.

Q-table của seed 42 được dùng trong demo theo quy định trước. Project không chọn seed đạt điểm cao nhất. Độ lệch chuẩn trong bảng tổng hợp là độ biến thiên giữa năm reward trung bình của các seed train, không phải khoảng tin cậy lâm sàng.

Phần 5

Cài đặt và giải thích mã nguồn

5.1 Tổ chức mã nguồn

Các module trong `diabetes_agent/` được dùng chung bởi notebook và giao diện. `data.py` đọc dữ liệu; `models.py` huấn luyện và suy luận; `knowledge.py` truy xuất tri thức; `rl.py` cài đặt môi trường và Q-learning; `system.py` quản lý thông điệp và điều phối. `app.py` nhận đầu vào, gọi hệ thống và hiển thị kết quả.

Các đoạn mã trong phần này được trích từ source của Project01. Tên tệp và số dòng được ghi phía trên mỗi đoạn; số dòng bên trái giữ theo tệp gốc.

5.2 Đọc dữ liệu và kiểm tra chia tập

Nguồn: `diabetes_agent/data.py`, dòng 11–26.

```
11 def load_data():
12     path = DATA / "classification.csv"
13     provenance = json.loads((DATA / "provenance.json").read_text(encoding="utf-8"))
14     digest = hashlib.sha256(path.read_bytes()).hexdigest()
15     if digest != provenance["study_sha256"]:
16         raise ValueError("Dataset checksum changed. Review provenance before retraining.")
17     frame = pd.read_csv(path, dtype={"group": str})
18     assert set(frame.target.unique()) == {0, 1}
19     assert not frame[FEATURES].isna().any().any()
20     groups = [set(frame.loc[frame.split == s, "group"])
21               for s in ("train", "validation", "test")]
22     assert all(not groups[i] & groups[j] for i in range(3) for j in range(i))
23     # Independently recompute feature fingerprints rather than trusting stored IDs.
24     hashes = pd.util.hash_pandas_object(frame[FEATURES], index=False)
25     assert frame.assign(fingerprint=hashes).groupby("fingerprint").split.nunique().max() == 1
26     return frame
```

Mã nguồn 5.1: Đọc CSV và kiểm tra tính toàn vẹn dữ liệu.

Hàm `load_data` đọc checksum từ `provenance.json`, tính SHA256 của CSV rồi so sánh trước khi nạp bảng. Hai phép kiểm tra sau đó xác nhận nhãn thuộc tập $\{0, 1\}$ và 21 cột đầu vào không có giá trị thiếu.

Biến `groups` chứa tập mã nhóm của train, validation và test. Phép giao giữa các tập được kiểm tra để phát hiện nhóm xuất hiện ở nhiều split. Phần cuối tính lại fingerprint từ chính 21 đặc trưng; nếu một fingerprint thuộc nhiều split, biểu thức `nunique().max()` lớn hơn 1 và chương trình dừng. Kết quả trả về là DataFrame đã qua các kiểm tra này.

5.3 Mạng MLP và chuẩn hóa đầu vào

Nguồn: `diabetes_agent/models.py`, dòng 28–30.

```
28 def architecture():
29     return nn.Sequential(nn.Linear(21, 64), nn.ReLU(), nn.Dropout(.15),
30                          nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 1))
```

Mã nguồn 5.2: Kiến trúc MLP sử dụng trong huấn luyện và suy luận.

`nn.Linear(21,64)` ánh xạ 21 chỉ báo vào lớp ẩn thứ nhất. ReLU tạo phi tuyến; dropout 0,15 được áp dụng khi huấn luyện. Lớp ẩn thứ hai có 32 nút, sau đó lớp cuối trả một logit. Sigmoid được áp dụng khi suy luận để tạo score trong khoảng $[0, 1]$.

Nguồn: `diabetes_agent/models.py`, dòng 78–88.

```
78     scaler = StandardScaler().fit(train[FEATURES])
79     x = torch.tensor(scaler.transform(train[FEATURES]), dtype=torch.float32)
80     xv = torch.tensor(scaler.transform(valid[FEATURES]), dtype=torch.float32)
81     y = torch.tensor(train.target.to_numpy(), dtype=torch.float32).reshape(-1, 1)
82     yv = torch.tensor(valid.target.to_numpy(), dtype=torch.float32).reshape(-1, 1)
83     net = architecture()
84     # Unweighted BCE keeps the observed training prior. No population calibration claim.
85     loss_fn = nn.BCEWithLogitsLoss()
86     optimizer = torch.optim.Adam(net.parameters(), lr=.001, weight_decay=1e-4)
87     best_loss, stale, best_epoch, best = float("inf"), 0, 0, None
88     history = []
```

Mã nguồn 5.3: Chuẩn hóa dữ liệu và khởi tạo bộ tối ưu.

Scaler chỉ được fit bằng `train[FEATURES]`; cùng scaler đó biến đổi train và validation. Tensor đặc trưng có 21 cột, còn `reshape(-1,1)` đưa nhãn về cùng dạng với đầu ra mạng. Loss là `BCEWithLogitsLoss`; Adam sử dụng learning rate 0,001 và weight decay 10^{-4}. Biến `best_loss` lưu loss validation thấp nhất, còn `stale` đếm số epoch liên tiếp không cải thiện.

5.4 Vòng huấn luyện và lưu checkpoint

Nguồn: `diabetes_agent/models.py`, dòng 89–113.

```
    for epoch in range(1, 81):
        net.train()
        order = torch.randperm(len(x))
        total = 0.
        for indexes in order.split(256):
            optimizer.zero_grad()
            loss = loss_fn(net(x[indexes]), y[indexes])
            loss.backward()
            optimizer.step()
            total += float(loss.item()) * len(indexes)
        net.eval()
        with torch.no_grad():
            val_loss = float(loss_fn(net(xv), yv).item())
        history.append({"epoch": epoch, "train_loss": total / len(x), "validation_loss":
 val_loss})
        if val_loss < best_loss - 1e-5:
            best_loss, stale, best_epoch = val_loss, 0, epoch
            best = copy.deepcopy(net.state_dict())
        else:
            stale += 1
        if stale >= 10:
            break
    net.load_state_dict(best)
    net.eval()
    torch.save(best, folder / "mlp.pt")
    joblib.dump(scaler, folder / "scaler.joblib")
```

Mã nguồn 5.4: Huấn luyện minibatch và chọn checkpoint theo validation.

Ở đầu mỗi epoch, `net.train()` bật chế độ huấn luyện và `randperm` xáo trộn thứ tự mẫu. Các chỉ số được chia thành minibatch 256 phần tử. Trong mỗi batch, gradient cũ được xóa, mạng tính logit, loss lan truyền ngược rồi Adam cập nhật tham số. Tổng loss được nhân với số mẫu của batch để tính trung bình theo mẫu.

Sau vòng minibatch, `net.eval()` tắt dropout; `torch.no_grad()` tính loss validation mà không lưu đồ thị gradient. Khi loss giảm hơn 10^{-5}, chương trình sao chép `state_dict` và đặt lại `stale`. Nếu 10 epoch liên tiếp không đạt mức cải thiện này, vòng lặp kết thúc. Mạng cuối cùng nạp lại checkpoint có loss validation thấp nhất; ở lần chạy báo cáo, checkpoint thuộc epoch 21.

5.5 Chọn ngưỡng và mô hình mặc định

Nguồn: `diabetes_agent/models.py`, dòng 33–36.

```
def choose_threshold(y, p):
    candidates = np.linspace(.05, .8, 151)
    # Ties resolve to the first threshold; all candidates are validation-only.
    return float(max(candidates, key=lambda t: f1_score(y, p >= t, zero_division=0)))
```

Mã nguồn 5.5: Chọn ngưỡng phân loại bằng F1 validation.

Hàm thử 151 ngưỡng từ 0,05 đến 0,80. Với từng ngưỡng t, dự đoán nhị phân là `p >= t`; hàm trả ngưỡng có F1 cao nhất. Thứ tự tăng dần của danh sách ngưỡng làm cho trường hợp bằng điểm chọn ngưỡng nhỏ hơn.

Nguồn: `diabetes_agent/models.py`, dòng 133–145.

```
133     validation = []
134     for name in [*models, "mlp"]:
135         p = probabilities(name, valid)
136         validation.append({"model": name, **metrics(valid.target, p, choose_threshold(valid.
        target, p))})
137     validation = pd.DataFrame(validation)
138     default = validation[validation.model != "dummy"].sort_values(
139         "average_precision", ascending=False).iloc[0].model
140     validation.to_csv(ARTIFACTS / "metrics_validation.csv", index=False)
141     manifest = {"seed": SEED, "default_model": default, "best_epoch": best_epoch,
142                 "features": FEATURES, "selection": "validation average precision; validation F1
        threshold",
143                 "thresholds": dict(zip(validation.model, validation.threshold)),
144                 "python": platform.python_version(), "torch": torch.__version__,
145                 "sklearn": sklearn.__version__, "architecture": "21 -> 64 ReLU -> 32 ReLU -> 1
        logit"}
```

Mã nguồn 5.6: Tổng hợp validation và lưu quyết định chọn mô hình.

Mỗi mô hình trả score trên validation, sau đó tính các chỉ số tại ngưỡng riêng. Dummy được giữ trong bảng đối chứng nhưng bị loại khỏi nhóm ứng viên mặc định. Các mô hình còn lại được sắp giảm dần theo Average Precision; tên mô hình đứng đầu và các ngưỡng được ghi vào manifest. Tập test được đánh giá ở bước sau bằng các quyết định đã lưu này.

5.6 Suy luận từ mô hình đã lưu

Nguồn: `diabetes_agent/models.py`, dòng 178–193.

```
178     def predict(self, profile, model="default"):
179         if model == "default":
180             model = self.manifest["default_model"]
181         if model not in self.manifest["thresholds"]:
182             raise ValueError("Unknown model")
183         frame = pd.DataFrame([profile])[FEATURES]
184         if model == "mlp":
185             with torch.no_grad():
186                 x = torch.tensor(self.scaler.transform(frame), dtype=torch.float32)
187                 score = float(torch.sigmoid(self.net(x)).item())
188         else:
189             score = float(self.models[model].predict_proba(frame)[0, 1])
190         threshold = self.manifest["thresholds"][model]
191         return {"model": model, "score": score, "threshold": threshold,
192                 "positive": score >= threshold,
193                 "meaning": "Điểm phân loại nhãn tiền tiểu đường/tiểu đường trên dữ liệu khảo sát
        UCI."}
```

Mã nguồn 5.7: Hàm dự đoán của lớp RiskModel.

Giá trị `default` được thay bằng tên trong manifest. Câu lệnh `[FEATURES]` cố định thứ tự 21 cột, tránh thay đổi vị trí đặc trưng khi hồ sơ JSON có thứ tự khóa khác. Nhánh MLP áp dụng scaler, tạo tensor, chạy mạng ở chế độ suy luận rồi lấy sigmoid; các mô hình sklearn trả cột xác

suất của lớp 1 qua `predict_proba`.

Ngưỡng được đọc theo đúng model ID. Hàm trả score, threshold và biến `positive` từ phép so sánh giữa hai giá trị. Các trường này được dùng trực tiếp để dựng vòng điểm và nhãn trên dashboard.

5.7 Truy xuất tri thức

Nguồn: `diabetes_agent/knowledge.py`, dòng 26–53.

```
26      def retrieve(self, query, k=3, graph=True):
27          q = normalize(query)
28          if not q.strip():
29              return []
30          lexical = cosine_similarity(self.vectorizer.transform([query]), self.matrix).ravel()
31          concepts = {n["id"] for n in self.graph["concepts"]
32                      if any(re.search(r"(?<!\w)" + re.escape(normalize(a)) + r"(?!\w)", q)
33                             for a in n["aliases"])}
34          expanded = set(concepts)
35          if graph:
36              for edge in self.graph["relations"]:
37                  if edge["source"] in concepts:
38                      expanded.add(edge["target"])
39          results = []
40          query_terms = set(re.findall(r"\w+", q)) - {"la", "gi", "va", "co", "khong", "toi", "can
    ", "the", "nao", "cua", "cho"}
41          for i, doc in enumerate(self.docs):
42              matched = set(doc["concepts"]) & expanded
43              bonus = .12 if graph and matched else 0.
44              # Do not let graph proximity create evidence for an unrelated query.
45              direct = bool(set(doc["concepts"]) & concepts)
46              anchor_terms = set(re.findall(r"\w+", normalize(doc["title"] + " " + " ".join(doc["
    keywords"])))))
47              if not direct and len(query_terms & anchor_terms) < 2:
48                  continue
49              if lexical[i] < .10 and not direct:
50                  continue
51              results.append({**doc, "score": float(lexical[i] + bonus),
52                              "matched_concepts": sorted(matched), "lexical_score": float(lexical[
    i])})
53          return sorted(results, key=lambda d: (-d["score"], d["id"]))[:k]
```

Mã nguồn 5.8: TF-IDF kết hợp mở rộng khái niệm một bước.

Hàm chuẩn hóa query rồi tính cosine similarity với ma trận TF-IDF của 12 ghi chú. Biến `concepts` chứa những khái niệm có alias khớp query theo ranh giới từ. Khi bật graph, các đích của cạnh đi ra từ tập này được thêm vào `expanded`; vòng mở rộng chỉ chạy trên tập ban đầu nên dừng ở một bước.

Với mỗi ghi chú, `matched` xác định các khái niệm giao với tập mở rộng. Bonus bằng 0,12 nếu có liên kết phù hợp. Bộ lọc yêu cầu khớp concept trực tiếp hoặc có ít nhất hai token trùng ở tiêu đề/từ khóa; cosine dưới 0,10 cũng bị loại nếu không khớp trực tiếp. Kết quả được sắp theo score giảm dần, dùng ID để xử lý hòa điểm và lấy tối đa $k = 3$ ghi chú.

5.8 Cập nhật bảng Q

Nguồn: `diabetes_agent/rl.py`, dòng 26–40.

```python
def train_q(seed=42, episodes=1800):
    rng = np.random.default_rng(seed)
    q = np.zeros((8, 3))
    history = []
    for episode in range(episodes):
        state, total = int(rng.integers(8)), 0.
        epsilon = max(.03, .8 * .997 ** episode)
        for day in range(28):
            action = int(rng.integers(3)) if rng.random() < epsilon else int(q[state].argmax())
            nxt, reward, _ = transition(state, action, rng)
            # The task is continuing. 28 days is a rollout window, not terminal disease state.
            q[state, action] += .08 * (reward + .92 * q[nxt].max() - q[state, action])
            state, total = nxt, total + reward
        history.append({"seed": seed, "episode": episode + 1, "reward_per_day": total / 28})
    return q, pd.DataFrame(history)
```

Mã nguồn 5.9: Huấn luyện Q-learning cho tác tử nhắc nhật ký.

Mảng `q` có kích thước 8×3, tương ứng tám trạng thái và ba hành động. Bộ sinh ngẫu nhiên được tạo từ seed truyền vào. Ở mỗi episode, tác tử chọn hành động ngẫu nhiên với xác suất epsilon; các lượt còn lại chọn `argmax` của hàng Q ứng với trạng thái hiện tại.

Hàm `transition` trả trạng thái kế tiếp và reward. Dòng cập nhật sử dụng learning rate 0,08, hệ số chiết khấu 0,92 và giá trị Q lớn nhất tại trạng thái kế tiếp. Sau 28 ngày, reward trung bình của episode được đưa vào history. Hàm trả cả bảng Q và DataFrame lịch sử; thí nghiệm gọi hàm với năm seed 42–46.

5.9 Giao tiếp và kiểm tra kết quả tác tử

Nguồn: `diabetes_agent/system.py`, dòng 70–81.

```python
    def send(self, sender, receiver, task, payload):
        self.trace.append({"sender": sender, "receiver": receiver, "performative": "REQUEST", "
    task": task})
        start = perf_counter()
        try:
            result = self.handlers[receiver](payload)
        except Exception as exc:
            result = {"status": "failure", "error": f"{type(exc).__name__}: {exc}"}
        self.trace.append({"sender": receiver, "receiver": sender,
                           "performative": "FAILURE" if result.get("status") == "failure" else "
    INFORM",
                           "task": task, "status": result.get("status", "completed"),
                           "elapsed_ms": round((perf_counter() - start) * 1000, 3)})
        return result
```

Mã nguồn 5.10: Gửi yêu cầu và ghi thông điệp phản hồi qua MessageBus.

MessageBus ghi một REQUEST trước khi gọi handler của bên nhận. Thời gian thực thi được đo bằng `perf_counter`. Kết quả thành công được ghi dưới dạng INFORM; exception được chuyển thành dữ liệu có trạng thái failure và thông điệp FAILURE. Lỗi tại một handler

vẫn có mặt trong trace để Coordinator và Council xử lý.

Nguồn: `diabetes_agent/system.py`, dòng 46–63.

```
46 def review(proposals, valid_source_ids):
47     """Council checks contracts, not clinical truth. It may veto a specialist."""
48     reasons = []
49     for item in proposals:
50         if item.get("kind") == "medication":
51             reasons.append("Không cho phép đề xuất thuốc hoặc liều điều trị.")
52         if item.get("status") == "failure":
53             reasons.append("Một tác tử không thực hiện được nhiệm vụ.")
54         if item.get("kind") == "knowledge":
55             evidence = item.get("evidence", [])
56             if item.get("status") == "completed" and not evidence:
57                 reasons.append("Kết luận tri thức không có bằng chứng.")
58             if any(d.get("id") not in valid_source_ids for d in evidence):
59                 reasons.append("Bằng chứng chứa mã nguồn không hợp lệ.")
60         if item.get("kind") == "risk" and "score" in item:
61             if not math.isfinite(item["score"]) or not 0 <= item["score"] <= 1:
62                 reasons.append("Điểm mô hình không hợp lệ.")
63     return {"status": "approved" if not reasons else "rejected", "reasons": reasons}
```

Mã nguồn 5.11: Các điều kiện kiểm tra của Council.

Council duyệt từng proposal, tích lũy lý do bác trong `reasons`. Các điều kiện gồm đề xuất loại medication, lỗi failure, kết quả tri thức thiếu bằng chứng hoặc chứa ID ngoài kho và score ngoài khoảng $[0, 1]$. Hàm chỉ trả approved khi danh sách lý do rỗng; các trường hợp còn lại trả rejected kèm lý do.

5.10 Điều phối một lượt xử lý

Nguồn: `diabetes_agent/system.py`, dòng 120–141.

```
120         safety = bus.send("Coordinator", "Intake", "CHECK_SCOPE", {"question": question, "
    context": context})
121         if safety["status"] != "completed":
122             result = {"status": safety["status"], "text": safety.get("text", safety.get("error",
     "Lỗi tiếp nhận")),
123                       "safety": safety, "proposals": [], "review": None}
124         else:
125             if task in ("consult", "risk"):
126                 proposals.append(bus.send("Coordinator", "Risk", "PREDICT_LABEL", {"profile":
    profile or {}, "model": model}))
127             if task in ("consult", "knowledge"):
128                 proposals.append(bus.send("Coordinator", "Knowledge", "RETRIEVE", {"question":
    question}))
129             if task in ("consult", "reminder"):
130                 proposals.append(bus.send("Coordinator", "Reminder", "PROPOSE_REMINDER", context
    ))
131             council = bus.send("Coordinator", "Council", "VERIFY", {"proposals": proposals})
132             if council["status"] != "approved":
133                 result = {"status": "rejected", "text": "Chưa thể tổng hợp: " + "; ".join(
    council["reasons"]),
134                           "proposals": proposals, "review": council}
135             else:
136                 advice = bus.send("Coordinator", "Advisor", "SUMMARIZE", {"proposals": proposals
    })
137                 statuses = {p.get("status") for p in proposals}
138                 result = {"status": "needs_input" if "needs_input" in statuses else
139                           ("insufficient_evidence" if "insufficient_evidence" in statuses else "
    completed"),
140                           "text": advice.get("text", advice.get("error", "Không thể tổng hợp")),
141                           "proposals": proposals, "review": council}
```

Mã nguồn 5.12: Nhánh tiếp nhận, gọi chuyên gia và tổng hợp của Coordinator.

Intake được gọi trước. Trạng thái khác completed tạo kết quả kết thúc với danh sách proposal rỗng. Nếu tiếp nhận thành công, tác vụ consult gọi cả Risk, Knowledge và Reminder; các tác vụ riêng chỉ gọi chuyên gia tương ứng. Tất cả proposal được chuyển tới Council trước khi Advisor tổng hợp.

Khi Council bác kết quả, trạng thái chung là rejected. Khi được duyệt, Advisor trả văn bản tổng hợp; trạng thái chung vẫn là needs_input nếu một chuyên gia yêu cầu bổ sung dữ liệu, hoặc insufficient_evidence nếu nhánh tri thức thiếu căn cứ. Việc duyệt cấu trúc proposal được tách khỏi tình trạng hoàn thành của nhiệm vụ.

5.11 Liên kết mã nguồn với giao diện

Nguồn: app.py, dòng 49–55.

```
49 def run_preset(task):
50     questions = {"consult": "HbA1c phản ánh điều gì và cần ghi nhật ký thế nào?",
51                  "knowledge": "HbA1c và thiếu máu có liên quan thế nào?", "reminder": "Nhắc tôi
       ghi nhật ký đường huyết"}
52     st.session_state.update(task=task, question=questions[task], profile_mode="Hồ sơ mẫu",
       use_glucose=False)
53     st.session_state.result = system.run(questions[task], sample_profile,
54         {"gap_days": st.session_state.gap, "fatigue": st.session_state.fatigue},
55         model=st.session_state.model, task=task)
```

Mã nguồn 5.13: Callback của ba kịch bản chạy nhanh.

Callback đặt tác vụ, câu hỏi và nguồn hồ sơ vào `st.session_state`, sau đó gọi `system.run`. Kết quả được lưu vào khóa `result`. Các kịch bản dùng cùng tham số nhắc việc và model đang được chọn trên giao diện, nên thao tác chạy nhanh đi qua cùng luồng xử lý với nút Bắt đầu phân tích.

Nguồn: app.py, dòng 58–69.

```
58 def render_flow(result):
59     responses = {m["sender"]: m for m in (result or {}).get("trace", []) if m["performative"] !=
        "REQUEST"}
60     tiles = []
61     for name, label, number in AGENTS:
62         status = responses.get(name, {}).get("status")
63         if name == "Coordinator" and result:
64             status = result["status"]
65         state = "done" if status in ("completed", "approved") else "attention" if status else "
       idle"
66         note = "Hoàn tất" if state == "done" else "Cần chú ý" if state == "attention" else "Sẵn
       sàng"
67         tiles.append(f'<div class="agent-node {state}"><span class="agent-number">{number}</span
       >'
68                      f'<strong>{label}</strong><small>{name}</small><span class="agent-state"><i
       ></i>{note}</span></div>')
69     st.markdown('<div class="agent-flow">' + "".join(tiles) + '</div>', unsafe_allow_html=True)
```

Mã nguồn 5.14: Dựng trạng thái hiển thị của bảy vai trò tác tử.

Hàm lấy các phản hồi trong trace theo tên tác tử và ánh xạ completed/approved sang trạng thái done. Các phản hồi khác được hiển thị attention; tác tử chưa có phản hồi giữ trạng thái idle. Coordinator dùng trạng thái tổng của phiên. Màu của các thẻ trên dashboard phản ánh vết xử lý đã ghi.

5.12 Notebook và các tệp kết quả

Bảng 5.1: Nội dung và đầu ra của bốn notebook.

Số	Nội dung	Đầu ra chính
01	Dữ liệu và MLP	Bộ chia tập, checkpoint, scaler, bảng validation/test và prediction từng dòng.
02	Tri thức và đồ thị	Danh mục ghi chú, ví dụ truy xuất và đánh giá 24 câu hỏi.
03	Q-learning	Bảng Q seed 42, lịch sử năm seed và so sánh policy.
04	Tích hợp hệ đa tác tử	Kết quả hội chẩn, trace, nhánh dữ liệu thiếu/cảnh báo và đánh giá hệ thống.

Bốn notebook có tổng cộng 26 code cell đã thực thi. Mô hình và các bảng số liệu được lưu trong `artifacts/`; nhật ký demo tùy chọn được lưu riêng tại `artifacts/demo_audit.sqlite3`.

Phần 6

Thực nghiệm và phân tích kết quả

6.1 Điều kiện thực nghiệm

Các thực nghiệm chạy trên CPU với Python 3.11.9, PyTorch 2.15.0+cpu, scikit-learn 1.9.1, pandas 2.3.3 và Streamlit 1.66.1. Phiên bản trực tiếp được ghi trong requirements, còn môi trường kiểm tra đầy đủ nằm ở `artifacts/environment.txt`.

Dữ liệu supervised dùng một split/seed cố định. RL chạy năm seed train và các seed eval tách biệt. Đánh giá phần mềm và truy xuất sử dụng kịch bản do tác giả xây. Test supervised đo phân loại nội bộ, simulator đo utility của policy và bộ tình huống kiểm tra hợp đồng phần mềm.

6.2 Kết quả validation và lựa chọn mặc định

Bảng 6.1: Chỉ số trên validation; ngưỡng được chọn bằng F1 validation.

Model	AUC	AP	F1	Precision	Recall	Brier
dummy	0.5000	0.1314	0.2322	0.1314	1.0000	0.1142
logistic	0.8127	0.3482	0.4350	0.3397	0.6047	0.0986
random_forest	0.8104	0.3713	0.4387	0.3383	0.6237	0.0967
mlp	0.8170	0.3706	0.4414	0.3285	0.6723	0.0960

Random Forest có Average Precision validation khoảng 0,3713, nhỉnh hơn MLP khoảng 0,3706. Theo quy tắc đã đặt, Random Forest được chọn làm model mặc định. MLP có ROC AUC validation cao hơn, nhưng tiêu chí chọn đã khóa là AP nên không đổi sang AUC sau khi thấy bảng kết quả.

Chênh lệch AP giữa hai model rất nhỏ và chưa có phân tích độ bất định để kết luận model nào tốt hơn ổn định. Lựa chọn mặc định chỉ là kết quả áp dụng quy tắc trong lần chia tập này. Giao diện vẫn cho chọn MLP để trình diễn nhánh học sâu và so sánh với các baseline.

6.3 Kết quả trên test đã khóa

Bảng 6.2: Chỉ số trên test; ngưỡng được chọn bằng F1 validation.

Model	AUC	AP	F1	Precision	Recall	Brier
dummy	0.5000	0.1373	0.2415	0.1373	1.0000	0.1185
logistic	0.8118	0.3820	0.4367	0.3499	0.5806	0.1003
random_forest	0.8096	0.3906	0.4534	0.3564	0.6230	0.0993
mlp	0.8168	0.3916	0.4431	0.3358	0.6512	0.0987

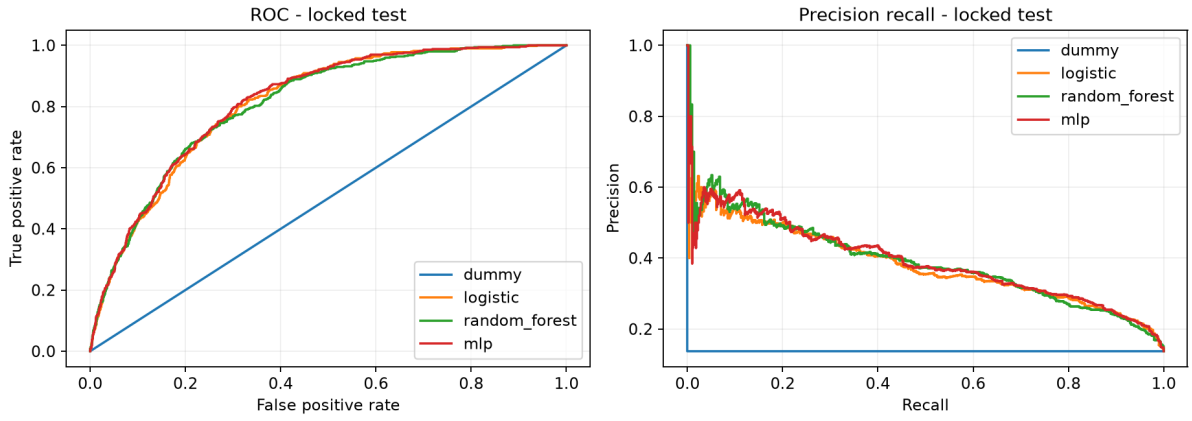
Bảng 6.3: Ngưỡng đã khóa và số lượng dự đoán trên test.

Model	Ngưỡng	TN	FP	FN	TP	Specificity
dummy	0.050	0	3116	0	496	0.0000
logistic	0.215	2581	535	208	288	0.8283
random_forest	0.230	2558	558	187	309	0.8209
mlp	0.205	2477	639	173	323	0.7949

MLP đạt ROC AUC khoảng 0,8168 và AP khoảng 0,3916; Random Forest đạt AUC khoảng 0,8096 và AP khoảng 0,3906. Logistic Regression đạt AUC khoảng 0,8118. Các model học được đều có AP cao hơn Dummy khoảng 0,1373, là tỷ lệ lớp dương trong test.

MLP nhỉnh hơn về AUC/AP trên test nhưng không làm thay đổi quyết định mặc định đã khóa từ validation. Các chênh lệch này là mô tả trên cùng split, chưa đủ để khẳng định ưu thế tổng quát.

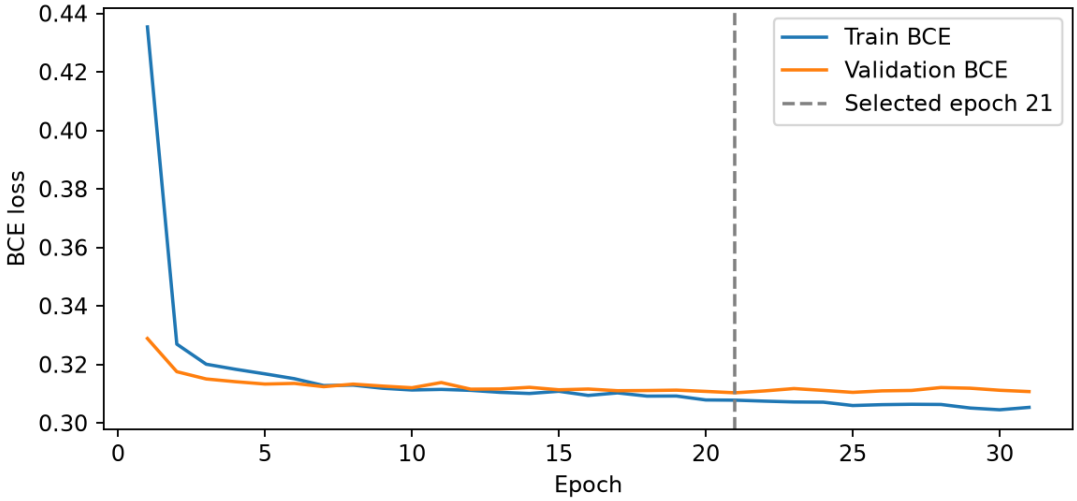
MLP phát hiện 323/496 mẫu lớp 1, còn bỏ sót 173 mẫu và tạo 639 dương tính giả. Random Forest phát hiện 309 mẫu, bỏ sót 187 và tạo 558 dương tính giả. Khi tăng recall, precision có thể giảm; việc chọn F1 validation không đồng nghĩa chọn ngưỡng phù hợp cho quyết định lâm sàng.



Hình 6.1: ROC và Precision–Recall trên test. Mọi quyết định về model/ngưỡng được thực hiện bằng validation trước đó.

Dummy được chọn ngưỡng nhỏ hơn điểm tiên nghiệm nên dự đoán tất cả là lớp 1, dẫn tới recall 1 và specificity 0. Precision, recall và ma trận nhầm lẫn trong các bảng phản ánh những mặt khác nhau của kết quả này.

6.4 Quá trình học của MLP



Hình 6.2: Loss train và validation, đường dọc thể hiện epoch được chọn để lưu checkpoint.

Checkpoint MLP được chọn tại epoch 21 theo BCE validation. Train tiếp tục cho tới khi đủ số epoch không cải thiện rồi nạp lại checkpoint tốt nhất. Ở các epoch cuối, train loss tiếp tục giảm trong khi validation loss tăng hoặc dao động.

Brier test của MLP khoảng 0,0987, thấp hơn Dummy khoảng 0,1185. Sai số score trên test nội bộ thấp hơn bộ dự đoán tiên nghiệm. Calibration theo nhóm và độ ổn định khi dịch chuyển phân bố chưa được đánh giá.

6.5 Truy xuất tri thức và vai trò đồ thị

Bảng 6.4: Đánh giá truy xuất trên 24 câu hỏi của bộ thực nghiệm.

Cấu hình	Hit@1	Hit@3	MRR
TF-IDF	1.0000	1.0000	1.0000
TF-IDF + graph	1.0000	1.0000	1.0000

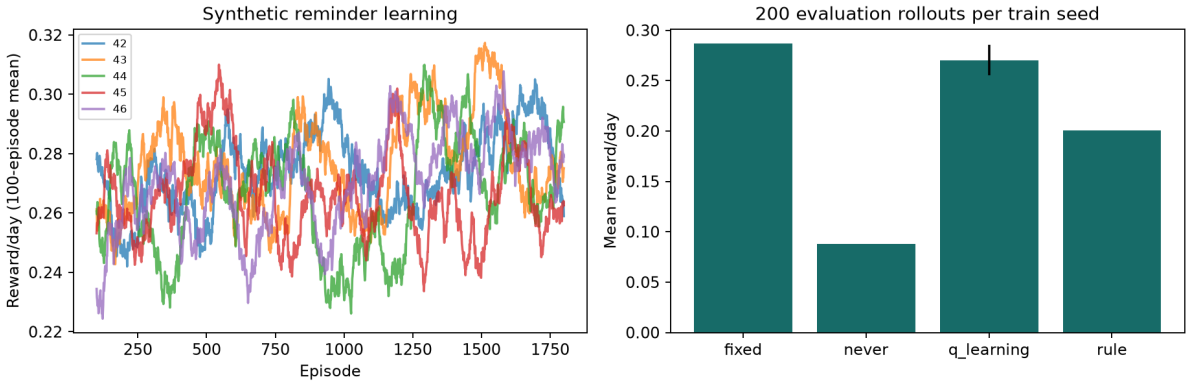
Cả hai cấu hình đều tìm đúng ID đầu tiên ở toàn bộ 24 câu trong phiên bản cuối. Vì vậy, bộ câu hiện tại không cho thấy lợi ích đo được của bonus đồ thị về Hit@1, Hit@3 hoặc MRR. Đồ thị vẫn có chức năng biểu diễn liên kết và mở rộng concept trong implementation, nhưng chưa có bằng chứng nó làm tăng chất lượng trên tập kiểm tra này.

Bộ đánh giá gồm 12 ID đích và 24 câu do tác giả viết, đã dùng trong phát triển. Chưa có bộ paraphrase độc lập hoặc đánh giá từ người dùng.

6.6 Kết quả Q-learning và đối chứng

Bảng 6.5: Reward mỗi ngày trong simulator; std giữa năm giá trị trung bình theo seed train.

Policy	Reward trung bình	Độ lệch chuẩn
fixed	0.2868	0.0000
never	0.0876	0.0000
q_learning	0.2703	0.0153
rule	0.2006	0.0000



Hình 6.3: Đường học năm seed và reward đánh giá trong môi trường nhắc việc giả lập.

Q-learning có reward trung bình khoảng 0,2703 mỗi ngày, cao hơn never khoảng 0,0876 và rule khoảng 0,2006, nhưng thấp hơn fixed khoảng 0,2868. Độ lệch chuẩn giữa năm seed train của Q-learning khoảng 0,0153.

Trong simulator này, lợi ích ghi nhận do hành động nhắc thường bù được chi phí nên policy luôn nhắc là đối chứng mạnh. Q-learning hữu hạn với learning rate cố định và năm seed chưa vượt đối chứng đó. Các policy được so sánh với cùng hàm reward.

Fixed/never/rule có std giữa seed train bằng 0 vì cùng policy được đánh giá lại với cùng 200 seed eval, không phải vì mọi rollout có reward giống nhau. Các dòng lặp của đối chứng không tạo thêm mẫu độc lập. Q-table seed 42 được nạp trong demo theo quy ước trước, không chọn theo reward eval.

6.7 Kiểm tra tích hợp và lỗi được inject

Bảng 6.6: Các kịch bản chức năng được chạy ở phiên bản bàn giao.

Kịch bản	Trạng thái mong đợi	Đạt	Thông điệp
risk_complete	completed	Có	8
risk_mlp	completed	Có	8
risk_missing	needs_input	Có	8
risk_invalid	needs_input	Có	8
risk_nan	needs_input	Có	8
knowledge_a1c	completed	Có	8
knowledge_food	completed	Có	8
knowledge_activity	completed	Có	8
knowledge_unknown	insufficient_evidence	Có	8
knowledge_blank	insufficient_evidence	Có	8
urgent_text	urgent	Có	2
urgent_seizure	urgent	Có	2
urgent_glucose	urgent	Có	2
invalid_unit_value	needs_input	Có	2
medication	out_of_scope	Có	2
reminder	completed	Có	8
invalid_reminder	needs_input	Có	8
consult	completed	Có	12
consult_missing	needs_input	Có	12
consult_unknown	insufficient_evidence	Có	12

Toàn bộ 20 kịch bản đạt trạng thái mong đợi ở phiên bản bàn giao. Kịch bản phủ luồng dự đoán, dữ liệu thiếu/sai mã, query có/không có bằng chứng, số đo bổ sung không hợp lệ, nhánh khẩn cấp, phạm vi thuốc và reminder. Các kịch bản kiểm tra những trạng thái đầu ra đã quy định.

Sáu proposal lỗi được inject riêng gồm thuốc, tri thức không nguồn, ID giả, score vượt khoảng, score NaN và lỗi handler. Council bác cả sáu. Ở cấu hình bỏ Council, các proposal được đánh dấu unreviewed. Phép ablation xác nhận chức năng veto đối với lỗi được dựng sẵn, chưa chứng minh MAS tốt hơn một pipeline đơn tác tử dùng cùng luật và công cụ.

6.8 Kiểm thử tự động và notebook

Bộ pytest có 32 trường hợp và đều đạt, bao gồm tính toàn vẹn dữ liệu, mã khảo sát, nạp model, truy xuất, ưu tiên urgent, Council veto, cô lập lỗi, Q-learning và lưu audit. Kiểm thử giao diện bằng Streamlit AppTest chạy hội chẩn, ba kịch bản nhanh, nhập JSON thiếu trường và xử lý glucose thấp.

Bốn notebook có lần lượt 7, 6, 5 và 8 code cell, tổng cộng 26 cell. Tất cả có execution count và không có error output. Log lưu tên file, số cell và thời gian thực thi. Giao diện còn được kiểm tra trực tiếp trong trình duyệt về bố cục và màu chữ.

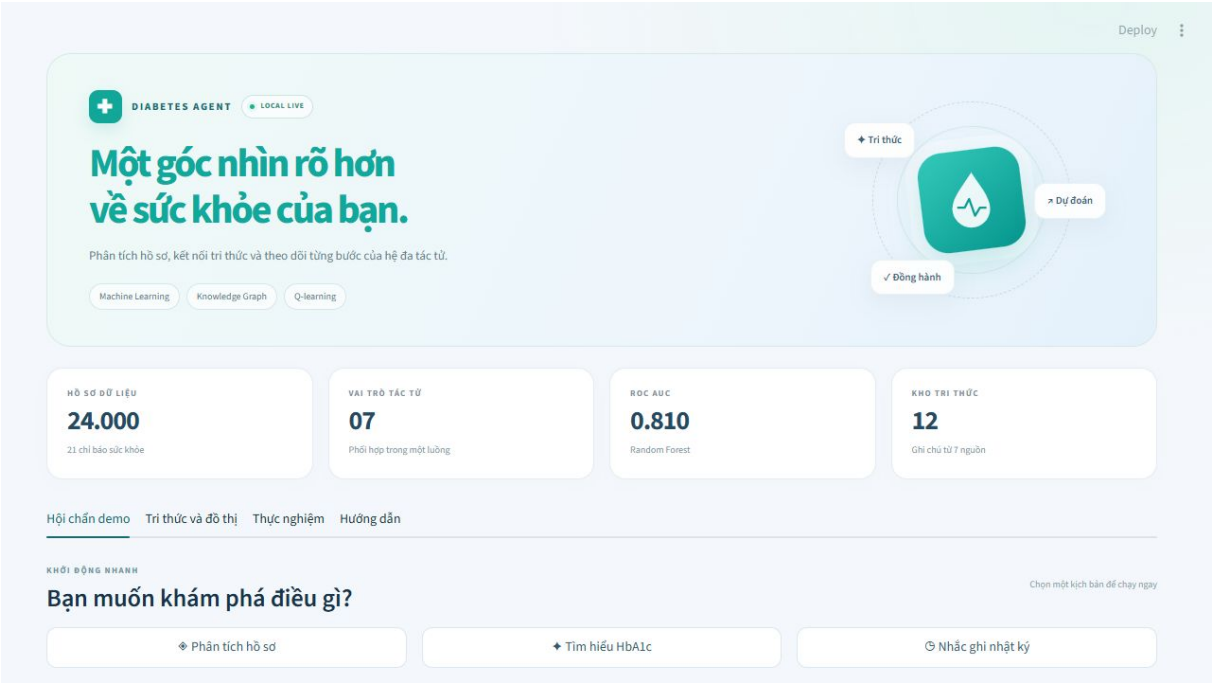
Phần 7

Giao diện và kết quả chạy demo

7.1 Dashboard tổng quan

Giao diện Streamlit tổ chức thành bốn tab: Hội chẩn demo, Tri thức và đồ thị, Thực nghiệm, Hướng dẫn. Phần đầu hiển thị tên hệ thống, quy mô dữ liệu, số tác tử và số ghi chú. Các nút chạy nhanh nạp đồng thời câu hỏi, hồ sơ và dữ liệu nhật ký của từng tình huống.

Trong tab hội chẩn, vùng nhập liệu gồm lựa chọn tác vụ, mô hình, câu hỏi và hồ sơ 21 chỉ báo. Hồ sơ mẫu lấy từ tập train; chế độ Nhập JSON cho phép thay toàn bộ vector đầu vào. Trạng thái của bảy tác tử được cập nhật từ thông điệp phát sinh khi chạy hệ thống.

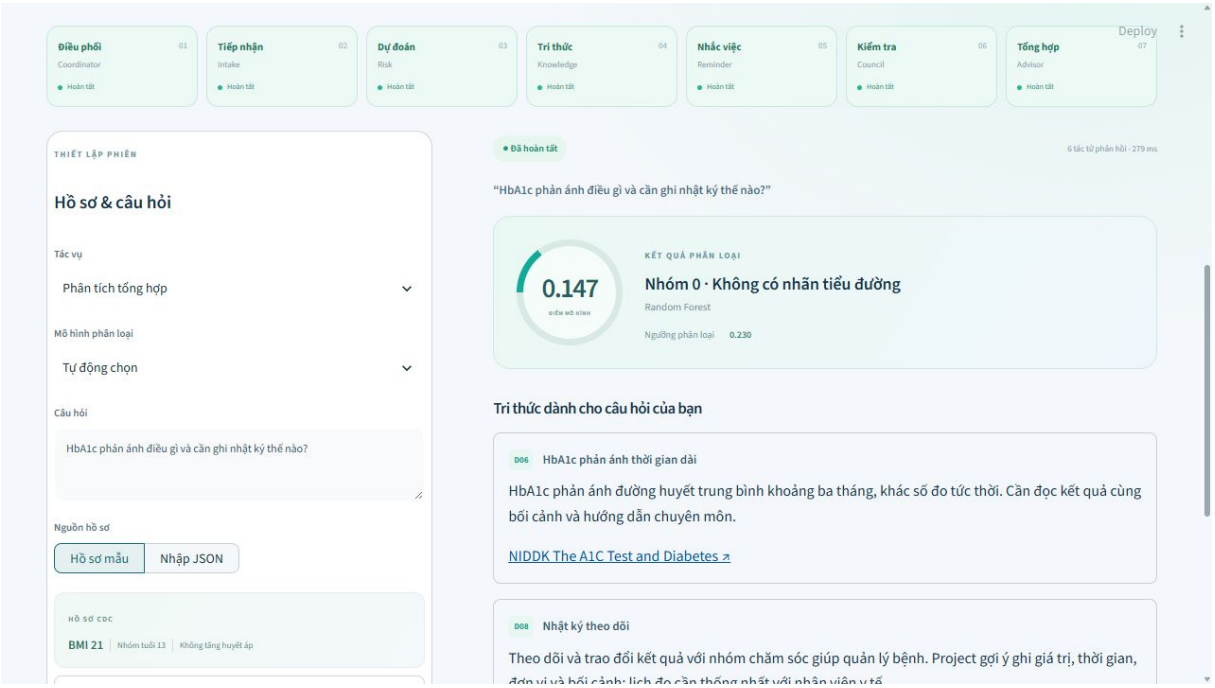


Hình 7.1: Dashboard tổng quan và các nút chạy nhanh của Diabetes Agent.

7.2 Kết quả phân loại và phối hợp tác tử

Ở lượt hội chẩn trong Hình 7.2, Random Forest trả score khoảng 0,147. Với ngưỡng 0,230 đã lưu trong artifact, kết quả thuộc lớp 0. Vòng điểm đi kèm tên mô hình và ngưỡng; các thẻ bên cạnh trình bày tri thức có nguồn và hành động nhắc nhật ký.

Luồng hoàn tất gồm Intake, ba nhánh chuyên môn, Council và Advisor, tạo 12 thông điệp request/response. Bảy thẻ trạng thái phía trên phản ánh các vai trò đã tham gia. Chi tiết phối hợp và nút tải JSON cung cấp proposal, evidence, trạng thái và thời gian xử lý của phiên.

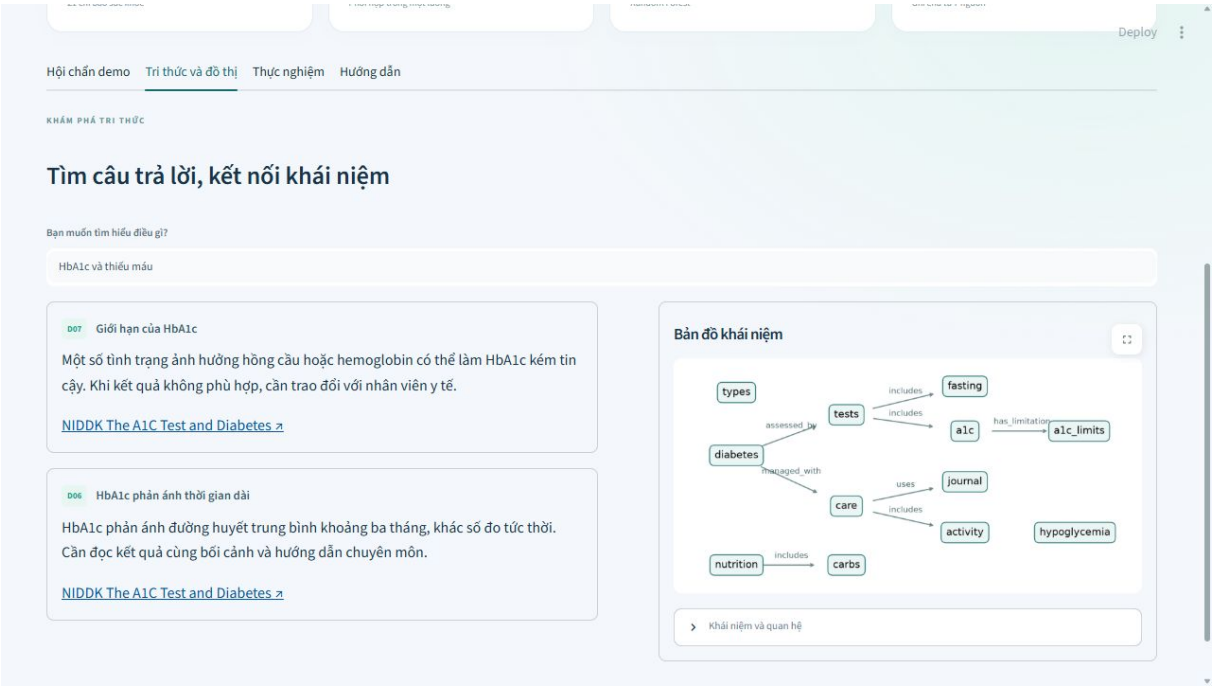


Hình 7.2: Kết quả hội chẩn với score, ngưỡng phân loại và trạng thái tác tử.

7.3 Tra cứu tri thức và bản đồ khái niệm

Truy vấn *HbA1c và thiếu máu* trả các ghi chú D07 và D06 trong ảnh chụp. Mỗi thẻ có ID, nội dung và liên kết nguồn. Bản đồ bên phải biểu diễn quan hệ giữa khái niệm xét nghiệm, HbA1c và các yếu tố ảnh hưởng đến kết quả xét nghiệm.

Tab này cho phép quan sát riêng nhánh Knowledge mà không chạy phân loại hồ sơ hoặc chính sách nhắc việc. Kết quả được lấy từ cùng bộ truy xuất dùng trong lượt hội chẩn.

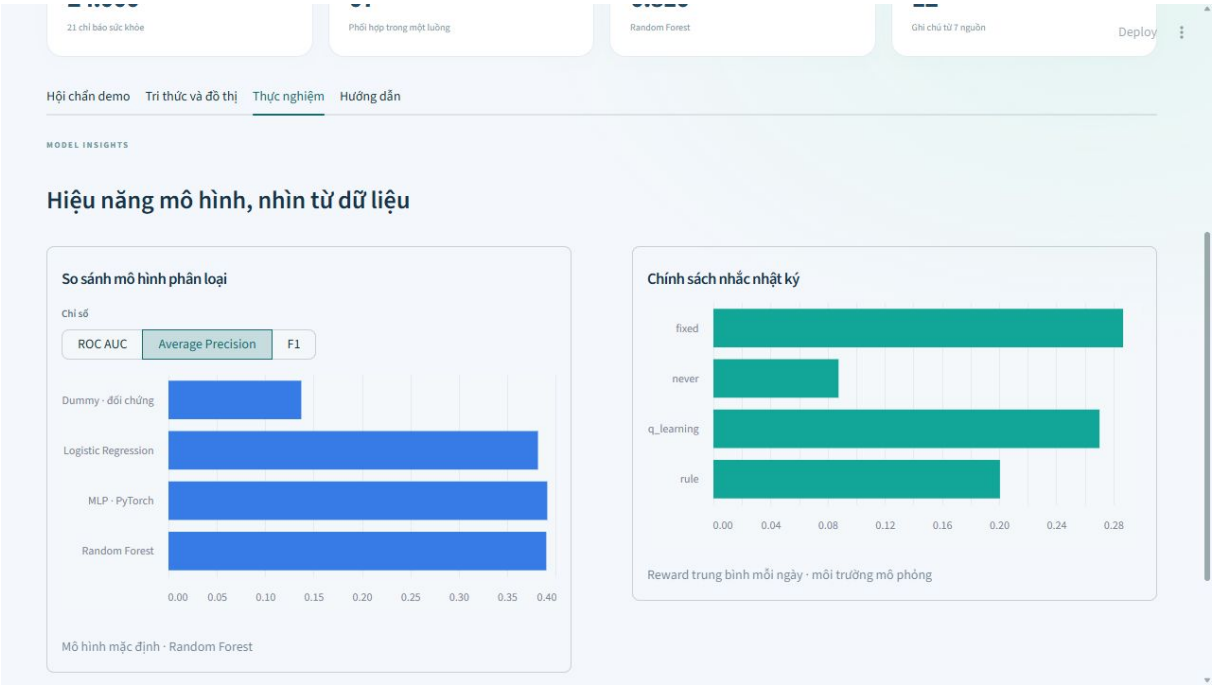


Hình 7.3: Kết quả truy xuất theo câu hỏi và đồ thị khái niệm trên giao diện.

7.4 Biểu đồ thực nghiệm trên giao diện

Tab Thực nghiệm đọc các bảng đánh giá đã lưu. Bộ chọn chỉ số thay đổi biểu đồ giữa ROC AUC, Average Precision và F1. Với AP test, MLP đạt khoảng 0,3916; Random Forest đạt 0,3906; Logistic Regression đạt 0,3820; Dummy đạt 0,1373.

Biểu đồ nhắc việc hiển thị reward của các policy trong cùng simulator: fixed khoảng 0,2868, Q-learning 0,2703, rule 0,2006 và never 0,0876. Các đường cong và bảng chi tiết phía dưới sử dụng cùng artifact với phần thực nghiệm của báo cáo.



Hình 7.4: Biểu đồ tương tác so sánh mô hình phân loại và chính sách nhắc việc.

Phần 8

Môi trường và cách chạy hệ thống

8.1 Cài đặt và khởi động local

Môi trường thực nghiệm sử dụng Python 3.11.9 64-bit trên CPU. Dữ liệu, kho tri thức, model, scaler và Q-table được đóng gói trong Project01. Thư viện được cài theo `requirements.txt`; danh sách phiên bản của môi trường thực nghiệm nằm trong `artifacts/environment.txt`.

Các lệnh sau chạy tại thư mục chứa `app.py` bằng PowerShell:

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install torch==2.15.0 --index-url https://
    download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m streamlit run app.py --server.address 127.0.0.1 --
    server.port 8521
```

Mã nguồn 8.1: Cài thư viện và khởi động giao diện local.

Giao diện mở tại http://127.0.0.1:8521. Lệnh Ctrl+C dừng server. Tệp `run.cmd` tự tạo venv và cài thư viện ở lần chạy đầu khi Python launcher tìm được Python 3.11. Sau khi cài thư viện, luồng suy luận sử dụng dữ liệu local và không yêu cầu API key.

8.2 Tái lập thực nghiệm và notebook

Các script chạy bằng Python của môi trường đã cài thư viện:

```
python scripts/run_experiments.py
python scripts/execute_notebooks.py
python -m pytest -q --junitxml=artifacts/test-results.xml
python scripts/update_report_assets.py
python scripts/export_report_code.py
```

Mã nguồn 8.2: Tái lập artifact, notebook và kiểm thử.

Script thực nghiệm hỗ trợ các stage models, rl và evaluation. Notebook 01 huấn luyện mô hình phân loại, notebook 02 thực hiện truy xuất, notebook 03 huấn luyện Q-learning, notebook 04 chạy tích hợp. Bốn notebook đã lưu output của tổng cộng 26 code cell. Môi trường Jupyter tương tác khởi động bằng `python -m notebook` tại thư mục project.

Quá trình train ghi lại artifact; giao diện nạp và cache các model khi chạy. Sau khi cập nhật artifact, việc khởi động lại server nạp phiên bản mới. Huấn luyện và demo được chạy lần lượt để tránh đọc tệp trong lúc ghi.

8.3 Dữ liệu và lưu kết quả

CSV là đầu vào huấn luyện. Bản SQLite chứa bảng `health_indicators` phục vụ truy vấn dữ liệu local. Kết quả phiên có thể tải dưới dạng JSON, gồm output của các tác tử và trace thông điệp. Chế độ audit tùy chọn lưu phiên vào SQLite với mã phiên và thời điểm UTC.

8.4 Các lỗi vận hành thường gặp

Bảng 8.1: Biểu hiện lỗi và thao tác khắc phục.

Biểu hiện	Khắc phục
Thiếu module	Cài requirements bằng đúng interpreter dùng để chạy app.
Lỗi nạp weight/joblib	Đối chiếu phiên bản môi trường; dùng môi trường tương thích hoặc huấn luyện lại từ CSV.
Thiếu manifest hoặc Q-table	Chạy stage thực nghiệm tương ứng để tạo lại artifact.
CSV khác checksum	Khôi phục CSV gốc trong bộ bàn giao.
Port 8521 đã được sử dụng	Đổi tham số `-server.port` và truy cập URL theo port mới.
Score chưa cập nhật sau train	Dừng và khởi động lại server để nạp model mới.

8.5 Biên dịch báo cáo

Source LaTeX nằm trong `report/`. Script `update_report_assets.py` cập nhật bảng và đồ thị từ artifact; `export_report_code.py` trích mã nguồn cùng số dòng vào báo cáo. XeLaTeX biên dịch hai lượt để cập nhật mục lục và tham chiếu. Các lệnh biên dịch được ghi trong `report/README.md`.

Phần 9

Kết luận và hướng phát triển

9.1 Kết quả đạt được

Project01 triển khai Diabetes Agent chạy local với ba nhánh phân loại hồ sơ, truy xuất tri thức và học chính sách nhắc nhật ký. Bảy vai trò phối hợp qua thông điệp, có kiểm tra đầu vào và duyệt proposal trước khi tổng hợp. Source, bốn notebook đã thực thi, dữ liệu, model và giao diện được đóng gói cùng báo cáo.

Random Forest được chọn theo AP validation 0,3713. Trên test, MLP đạt AP 0,3916 và Random Forest đạt 0,3906, đều cao hơn Dummy 0,1373. Hai cấu hình truy xuất đạt Hit@1 bằng 1 trên 24 câu tự xây; bonus đồ thị chưa tạo chênh lệch trong bộ câu này. Q-learning đạt reward 0,2703 mỗi ngày, cao hơn never và rule nhưng thấp hơn fixed 0,2868. Hệ thống đạt 20/20 kịch bản tích hợp và 32/32 kiểm thử tự động.

9.2 Giới hạn và hướng phát triển

Phân loại mới được đánh giá trên một split/seed của dữ liệu khảo sát Hoa Kỳ với nhãn gộp; chưa có đánh giá ngoài mẫu, theo nhóm hoặc khoảng tin cậy. Các thử nghiệm tiếp theo gồm lặp split/seed, so sánh mã hóa biến nhân khẩu và đánh giá calibration trên dữ liệu độc lập.

Kho tri thức có 12 ghi chú; câu hỏi truy xuất do tác giả biên soạn và đã được sử dụng trong phát triển. Bộ câu độc lập với cách diễn đạt đa dạng và đánh giá thủ công độ liên quan sẽ bổ sung bằng chứng cho nhánh này. Hiệu quả kiến trúc đa tác tử so với pipeline đơn dùng cùng công cụ cũng chưa được đo.

Nhánh RL sử dụng động lực phản ứng và reward giả lập. Phân tích độ nhạy theo hệ số reward, đối chiếu với nghiệm tối ưu của MDP nhỏ và đánh giá trên dữ liệu hành vi là các hướng mở rộng. Những kết quả hiện tại phản ánh chất lượng chính sách trong simulator đã định nghĩa.

Tài liệu tham khảo

[1] M. Wooldridge và N. R. Jennings, *Intelligent agents Theory and practice*, The Knowledge Engineering Review, 10(2), 1995. https://www.cs.ox.ac.uk/people/michael.wooldridge/pubs/ker95.pdf.

[2] C. J. C. H. Watkins và P. Dayan, *Q-learning*, Machine Learning, 8, 279–292, 1992. https://www.gatsby.ucl.ac.uk/~dayan/papers/cjch.pdf.

[3] UCI Machine Learning Repository, *CDC Diabetes Health Indicators*, dataset 891, metadata và CSV. https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators. Kiểm tra 05/11/2026.

[4] PyTorch, *BCEWithLogitsLoss documentation*. https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html. Kiểm tra 05/11/2026.

[5] scikit-learn, *average_precision_score documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html. Kiểm tra 05/11/2026.

[6] NIDDK, *What Is Diabetes*. https://www.niddk.nih.gov/health-information/diabetes/overview/what-is-diabetes. Kiểm tra 05/11/2026.

[7] NIDDK, *Diabetes Tests and Diagnosis*. https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis. Kiểm tra 05/11/2026.

[8] NIDDK, *The A1C Test and Diabetes*. https://www.niddk.nih.gov/health-information/diagnostic-tests/a1c-test. Kiểm tra 05/11/2026.

[9] NIDDK, *Managing Diabetes*. https://www.niddk.nih.gov/health-information/diabetes/overview/managing-diabetes. Kiểm tra 05/11/2026.

[10] CDC, *Treatment of Low Blood Sugar Hypoglycemia*. https://www.cdc.gov/diabetes/treatment/treatment-low-blood-sugar-hypoglycemia.html. Kiểm tra 05/11/2026.

[11] CDC, *Diabetes Meal Planning*. https://www.cdc.gov/diabetes/healthy-eating/diabetes-meal-planning.html. Kiểm tra 05/11/2026.

[12] CDC, *Get Active*. https://www.cdc.gov/diabetes/living-with/physical-activity.html. Kiểm tra 05/11/2026.

[13] Trần Văn Khánh, *Báo cáo Project01 Hệ đa agent hỗ trợ sàng lọc và tư vấn bệnh tiểu đường*, 36 trang, tháng 11/2026.